

Repo Audit & Reorganization

Prompted by a real GitHub Actions failure email, not a scheduled review — what the audit found, the folder restructure that came out of it, and every reference that had to be updated to keep it working. Standalone, not tied to a numbered build.

STATUS

Complete — repo reorganized, CI re-verified green

TRIGGER

A CI failure email after the first push of Build 6 Phase 2 onward

FOLLOWS

Build 7 (verifier/critic agent)

[What Triggered This](#)[The CI Bug](#)[Full Audit Findings](#)[The Reorganization](#)[Verification](#)

What Triggered This

Not a scheduled review — a GitHub Actions failure email arrived after a routine push (stripping a co-author trailer from git history). Checking it against real run data surfaced something bigger than the immediate failure: `origin/main` had been 23 commits behind local for a while. All of Build 6 Phase 2 onward, and all of Build 7, had been built and committed locally but never pushed — that push was the first time any of it reached CI at all.

The lesson generalizes past this one incident. A green CI badge only means something if CI actually ran against the code being described. Work sitting local-only, however well-tested by hand, isn't verified by a CI pipeline that's never seen it.

The CI Bug Itself

CI correctly went red. 25 of 27 tests passed; `test_chat_rounds.py`'s two DB-touching tests failed with relation `"agent_scratch.chat_rounds"` does not exist.

Root cause: `ci.yml`'s migration step is a hardcoded file list. `012_chat_rounds_schema.sql` (Build 6 Phase 3.5, persistent chat history) was added to the repo but never added to that list — in either the `test` job or the `eval` job. Same bug class as `extract_doc_chunks.py`'s old hand-maintained `SOURCE_FILES` list, which had already caused two separate incidents earlier in the project before being fixed with directory-scan auto-discovery.

Fixed: both migration lists now include `012_chat_rounds_schema.sql`. Confirmed via a real `gh run watch` to a fully green run, not assumed from the diff alone.

Full Audit Findings

The CI bug prompted a wider question: what else has drifted the same way?

Checked every hardcoded list, generated-vs-real-file relationship, and config-vs-code dependency in the project.

Build 7's verifier eval harness never ran in CI.

`run_verify_eval.py/verify_eval_cases.py` weren't invoked anywhere in `ci.yml`, and `verify_answer.py` wasn't in the `detect-changes` paths-filter — a change to the verifier's own logic wouldn't even re-trigger the eval job. Fixed: both added.

.env.example documented 4 of the 10 env vars the code actually requires.

`ANTHROPIC_API_KEY` and both reader/writer role credentials were entirely absent. `tools.py` reads most of these with no default (`os.environ[...]`), so a fresh clone following the file as written hits an immediate `KeyError`. Rewritten with all 10, each with a one-line note on why it's needed.

No README.md existed.

A repo whose entire purpose is a portfolio artifact had no entry point for a visitor landing on it. Added.

db_test_py.py — a stray early debugging script.

Same spirit as Build 5's deleted `api_test.py`: an early Postgres connectivity check querying a `customers` table no longer in the agent's actual scope. Its filename doesn't match either of pytest's discovery globs, so it was never at risk of silent collection — but it was dead, untested code sitting in the project root. Deleted.

An empty VS Code scratch file was tracked in git.

Local `Postgres.session.sql`, 0 bytes, auto-created by a SQL client extension. Deleted.

Two vestigial SQL Server files, unreferenced by anything.

`schema_mssql.sql/seed_mssql.sql` — part of the original dual-database lab environment, not connected to any code, CI step, or Docker config in the actual seven-build narrative. Relocated to `legacy/` rather than deleted, since `docker-compose.yml`'s `mssql_local` service still exists.

Verified clean, not assumed: `Dockerfile`'s `COPY` list against the actual file set, `requirements.txt` against every top-level import actually used, `.gitignore` coverage (including confirming `.venv/` was never at risk — it has its own internal `.gitignore`), and `extract_doc_chunks.py`'s glob-based doc discovery (already fixed correctly in an earlier build, confirmed not regressed).

The Reorganization

A flat ~50-file repo root had never been restructured as the project grew across seven builds. Before moving anything, the actual Python import graph was checked directly — `run_eval.py/run_verify_eval.py` import `agent/verify_answer` directly, and every test file imports `tools/agent/logging_utils` directly. That ruled out splitting "core app" from "pipeline/eval scripts" into separate directories without adding `sys.path` configuration everywhere — so every first-party `.py` file stays together, flat, in one directory.

FOLDER	CONTENTS
<code>src/</code>	All 22 first-party Python modules and scripts, flat — required by the import graph above.
<code>migrations/</code>	The 12 numbered SQL migrations, unchanged content.
<code>tests/</code>	All 7 <code>test_*.py</code> files. <code>pytest.ini</code> gained <code>testpaths = tests</code> and <code>pythonpath = src</code> so <code>import tools</code> etc. keeps resolving with zero changes to the test files themselves.
<code>docs/</code>	Renamed from <code>PROJECT_DIAGRAMS/</code> . Every per-build diagram/brief folder, plus the cheat sheet, moved in.
<code>legacy/</code>	The two vestigial SQL Server files.
<code>data/</code>	Unchanged location — already its own folder.

Every reference updated, not just the files themselves: `Dockerfile` (`COPY/CMD` now target `src/`), `docker-compose.yml` (`streamlit_app`'s command), `ci.yml` (both migration-loop path prefixes, every `python script.py` invocation, the `paths-filter` list), `extract_doc_chunks.py` (its `glob` target and hardcoded seed-file paths, since it reads `docs/` by relative path from the repo root regardless of its own new location in `src/`), and `.gitignore` (the personal working-style folder's path).

All moves done via `git mv`, not delete-and-recreate. Git records each as a rename, preserving `git log --follow` history for every file rather than starting it over.

One known follow-up, not yet done: `pg_local`'s `docs.chunks` table still has rows keyed by the old `PROJECT_DIAGRAMS/...` source-file strings. The next time `extract_doc_chunks.py` runs locally, those become orphaned duplicates alongside the new `docs/...`-keyed rows — same failure shape as the backslash/forward-slash duplicate bug from Build 6, different cause. Fix when next touching the docs-RAG pipeline locally: `DELETE FROM docs.chunks WHERE source_file LIKE 'PROJECT_DIAGRAMS/%' OR source_file = 'sql-test-01-cheatsheet.html'` before re-running the extract/embed pair. CI is unaffected — it always starts from an empty database.

Verification

py_compile across every moved file

Clean on all 22 `src/` files and all 7 `tests/` files.

Full Docker image build

Built against the reorganized paths end to end before committing, not assumed from reading the Dockerfile diff. Confirmed inside the running container (not just a successful build exit code) that `agent.py` and `verify_answer.py` both resolve every import correctly from their new `src/` location.

Live CI run, watched to completion

`detect-changes` ✓, `test` ✓ (all 27) — confirmed via `gh run watch`, the same live-verification standard every prior build in this project has used.

Cheat sheet re-verified both ways

`--screenshot` and `--print-to-pdf/pdftoppm`, after every path reference across the Project Files table was rewritten to match the new structure.

The reorg's own CI run found one more real bug, on the first genuine eval run since Build 6. Because this commit's rename touched `src/agent.py` (a paths-filter match), `eval` actually ran — not gated out — for the first time since `log_final_answer` was added weeks earlier. It failed immediately: `run_eval.py`'s `tools_called = sorted({entry["tool_name"] for entry in new_entries})` assumes every log line appended during a question is a tool-call entry. It isn't — every question has logged a `log_final_answer` entry (no `tool_name` key) since Build 6, and a `log_verification` entry since Build 7, both sharing the same `tool_calls.jsonl` file. `KeyError: 'tool_name', every time.`

Fixed, commit 071f784: added the same `"tool_name"` in `entry guard` `logging_utils.get_tool_calls()` and `view_recent_questions.py` already use elsewhere in this codebase — this file just never got the memo when the log gained new entry shapes.

The fix commit itself then didn't get re-verified by CI — a second, related gap. A push touching only `src/run_eval.py` doesn't match the paths-filter (`agent.py/tools.py/verify_answer.py/the case files` — never the runner scripts that actually execute them), so `eval` correctly stayed gated on that push, silently. The fix would have gone unverified by CI entirely.

Fixed two ways, commit eec4707: `src/run_eval.py` and `src/run_verify_eval.py` added to the paths-filter, so a change to either runner now correctly re-triggers `eval`. And rather than push again and wait on a third CI round trip, both were verified directly against `pg_local` first — `run_eval.py` 6/7 (the one failure is a pre-existing

eval case about "order revenue," a concept that isn't part of this project's actual domain anymore, unrelated to either fix) and `run_verify_eval.py` 3/3, both with real API calls, before pushing.

Why this matters beyond the two one-line fixes: the original bug had been sitting live in `main` for roughly two weeks, silently unverified, because nothing had reached `origin/main` since before it was introduced. A green CI badge that hasn't actually run recently isn't verifying anything — the same lesson this whole maintenance pass started from, twice over in one afternoon.

Repo audit and reorganization complete — CI green against the real, current file structure, including a genuinely new bug the reorg's own CI run caught and this document then fixed. Ongoing cross-build command reference lives in `docs/sql-test-01-cheatsheet.html`.